MONTH 3 PROJECT REPORT

WEATHER DATA PIPELINE SYSTEM

A Python-based ETL, storage, analysis and reporting application

Submission detail	Information
Student name	SAGAR MISHRA
Course / program	INTERNSHIP
Institution	DEVELOPERS ARENA
Submission date	04 November 2026

Prepared for Database Management & APIs

Report Preparation Note

This report is based on the project structure, README, database schema and execution results shared during development. Replace bracketed student details before submission. The APScheduler installation was still in progress when the report was prepared; therefore, successful automated scheduling is marked as needing verification. Add the real captured screenshots in Appendix B.

Contents

- 1. Executive Summary
- 2. Project Overview and Objectives
- 3. Tools and Technologies
- 4. System Architecture
- 5. Database Design
- 6. ETL Workflow and Data Quality
- 7. API Integration
- 8. Analysis, Reports and Alerts
- 9. Monitoring and Logging
- 10. Testing and Results
- 11. Setup and Execution
- 12. Deployment and Security
- 13. Challenges and Limitations
- 14. Conclusion and Future Enhancements
- Appendix A. Project Structure
- Appendix B. Screenshot Evidence
- Appendix C. Requirement Mapping

1. Executive Summary

The Weather Data Pipeline System is a Python application that retrieves current weather observations from the OpenWeatherMap API, validates and transforms the data, stores it in a normalized SQLite database, and provides command-line tools for reports, historical summaries, statistics, health checks and run history.

In the demonstrated successful run, the pipeline requested data for five cities—Mumbai, Delhi, Bengaluru, Chennai and Kolkata. All five requests succeeded; five records were inserted; there were no failures, duplicates or errors. The health check reported a healthy database, five tracked cities, fifteen observations, zero failed runs and data that was not stale. The test suite reported five passing tests.

2. Project Overview and Objectives

The project provides a repeatable process for collecting weather information from an external API and organizing it for storage, inspection and reporting. Time-stamped observations can accumulate over repeated collection runs and support historical summaries.

2.1 Objectives

- Connect to the OpenWeatherMap real-time weather API.
- Collect observations for multiple configured cities.
- Validate required fields, timestamps and plausible measurement ranges.

Month 3: Database Management & APIs

- Load clean observations into a relational SQLite database.
- Prevent duplicate city-and-observation records.
- Record pipeline execution status and errors.
- Generate JSON and CSV weather reports.
- Provide historical query, statistics and health-check commands.
- Support recurring automated collection through a scheduler.

3. Tools and Technologies

Technology	Role
Python	Main language for API access, ETL, CLI and reporting.
OpenWeatherMap API	External source of current weather observations.
SQLite	Relational storage for cities, observations and pipeline runs.
pytest	Automated testing framework.
APScheduler	Dependency used by the scheduling feature; runtime verification is pending.
VS Code and PowerShell	Development environment on Windows.
CSV and JSON	Machine-readable report formats.

4. System Architecture

The application separates API communication, processing, persistence, reporting and monitoring into modules.

- src/main.py — command-line entry point for run, schedule, report, stats, health and history.
- src/api_client.py — communicates with OpenWeatherMap.
- src/etl_pipeline.py — coordinates extraction, transformation, validation and loading.
- src/validators.py — validates fields, timestamps and measurement ranges.
- src/database.py — manages SQLite operations.
- src/reporter.py — prepares historical summaries, alerts and JSON/CSV reports.
- src/scheduler.py — recurring collection workflow.
- src/monitor.py and src/logging_setup.py — health checks and logging.

High-level data flow: OpenWeatherMap API → API client → ETL transformation and validation → SQLite database → historical queries and reports → health checks and logs.

5. Database Design

The SQLite database uses three related tables to separate city reference data, time-stamped weather measurements and pipeline execution metadata.

Table	Purpose	Key fields and constraints
cities	Configured city reference data.	city_id primary key; city_name; country_code; latitude; longitude; unique city_name + country_code.
weather_observations	Weather measurements for each city.	observation_id primary key; city_id foreign key; observed_at; fetched_at; temperature; humidity; pressure; wind; weather description; unique

		city_id + observed_at.
pipeline_runs	Pipeline execution history.	run_id primary key; optional city_id foreign key; started_at; completed_at; status; records_processed; error_message.

Indexes support city/time observation access and pipeline run start-time queries. A CHECK constraint restricts run status to running, success, failed or skipped.

5.1 Relationships

- One city can have many weather observations.
- A pipeline run can be associated with a city; its city reference is nullable.
- Foreign keys connect observations and applicable pipeline runs to cities.

6. ETL Workflow and Data Quality

Extract: The API client requests current weather data from OpenWeatherMap for each configured city.

Transform: API response fields are mapped into a consistent observation structure with observation and fetch timestamps.

Validate: Checks include required fields, ISO timestamps, plausible temperature, humidity bounds, and nonnegative pressure and wind values.

Load: Validated records are written to SQLite. A uniqueness rule on city and observation timestamp helps prevent duplicates.

Error handling and logging provide visibility into API or processing failures. Pipeline run records retain execution status and record counts.

7. API Integration

The application uses the OpenWeatherMap current-weather API as its external source. The API key is configured locally through environment variables loaded from a .env file. The key must never be uploaded to a public repository. Use .env.example as a template containing placeholders only. Endpoint parameters and response mapping are documented in docs/api_guide.md.

8. Analysis, Reports and Alerts

The report command generates JSON and CSV outputs containing latest weather information, historical summaries and alerts. The demonstrated report included a high-humidity alert for Kolkata at 91%.

The history and stats commands support inspection of stored measurements. Current-weather API responses are point-in-time observations rather than historical backfill. Meaningful 30-day analysis requires regular collection over that period or a separate historical dataset.

The available evidence confirms city summaries and an alert, but does not establish that all proposed analysis questions—such as humidity/rainfall correlation, seasonal extremes or peak temperature hours—are implemented. Verify the relevant queries and results before claiming them.

9. Monitoring and Logging

The health command checks database connectivity, tracked cities, observation count, failed pipeline runs and data freshness. The README identifies logs/pipeline.log as the default log file.

Demonstrated health output: database = ok; cities_tracked = 5; observations = 15; failed_runs = 0; data_stale = false; status = healthy.

10. Testing and Results

Command: `python -m pytest -q`

Observed result: 5 passed (approximately 0.4 seconds).

10.1 Demonstrated pipeline run

Metric	Observed result
Cities requested	5
Successful requests	5
Failed requests	0
Records inserted	5
Duplicates	0
Errors	None

These are the results shared during development. Later runs may differ depending on API availability, configuration and returned data.

11. Setup and Execution

The following steps apply to the Windows PowerShell environment:

1. Install Python 3.10 or later.
2. Create a virtual environment: python -m venv .venv
3. Activate it: .venv\Scripts\activate
4. Install dependencies: python -m pip install -r requirements.txt
5. Create .env from .env.example and add the OpenWeatherMap API key locally.
6. Run collection: python -m src.main run
7. Generate reports: python -m src.main report
8. Check health: python -m src.main health
9. Run tests: python -m pytest -q

11.1 CLI Commands

- python -m src.main run — collect current weather observations.
- python -m src.main schedule — start recurring collection; verify after APScheduler installation.
- python -m src.main report — generate JSON and CSV reports.
- python -m src.main stats — display pipeline/data statistics.
- python -m src.main health — display database and pipeline health.
- python -m src.main history --city Mumbai --days 30 — inspect stored history for a city.

12. Deployment and Security

For deployment, install the required Python version and dependencies in an isolated environment, configure the API key securely, initialize the database schema, and run the application using its documented CLI. A production setup should use a reliable process supervisor or scheduled service and monitor logs and backups.

For GitHub, include source code, documentation, tests, scripts, requirements.txt, .env.example and .gitignore. Exclude .env, .venv, __pycache__, .pytest_cache and private credentials. Include only non-sensitive sample reports and logs if required.

13. Challenges and Limitations

- The schedule command previously failed with ModuleNotFoundError: No module named 'apscheduler'. Installation was being attempted; successful scheduling has not yet been confirmed.
- Current-weather API responses are not a historical backfill; trend analysis depends on collecting observations over time.
- API availability, network connectivity, rate limits and credential configuration can affect extraction.
- The demonstrated automated test suite contains five passing tests; broader integration and edge-case coverage could be added.

14. Conclusion and Future Enhancements

The project demonstrates an end-to-end foundation for collecting current weather data, validating it, storing it in a relational database and generating reports. The demonstrated five-city run succeeded, the health check reported a healthy database and all five automated tests passed.

- Verify recurring scheduling after resolving the APScheduler dependency.
- Add a dashboard with charts for temperature, humidity, pressure and wind trends.
- Implement and test explicit queries for the assignment's analysis questions.
- Expand integration tests and simulate API failures.
- Automate database backups and report retention.
- Document deployment through a scheduled Windows task or hosted environment.

Appendix A. Project Structure

```
weather-data-pipeline/
├── config/
├── database/  (schema.sql and local SQLite database)
├── docs/      (api_guide.md, architecture.md, database_schema.md, troubleshooting.md)
├── logs/
├── reports/
├── screenshots/
├── scripts/   (backup_database.py)
├── src/       (api_client.py, database.py, etl_pipeline.py, logging_setup.py,
│               main.py, monitor.py, reporter.py, scheduler.py, validators.py)
├── tests/     (test_pipeline.py, test_validators.py)
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

Appendix B. Screenshot Evidence

Insert the actual screenshots captured from the local project in the spaces below, or keep them in the screenshots/ folder and reference them here. The screenshot image files were not available to embed automatically in this document.

Figure B1. Successful pipeline run

Show five requested, five succeeded, five inserted, zero failed and no errors.

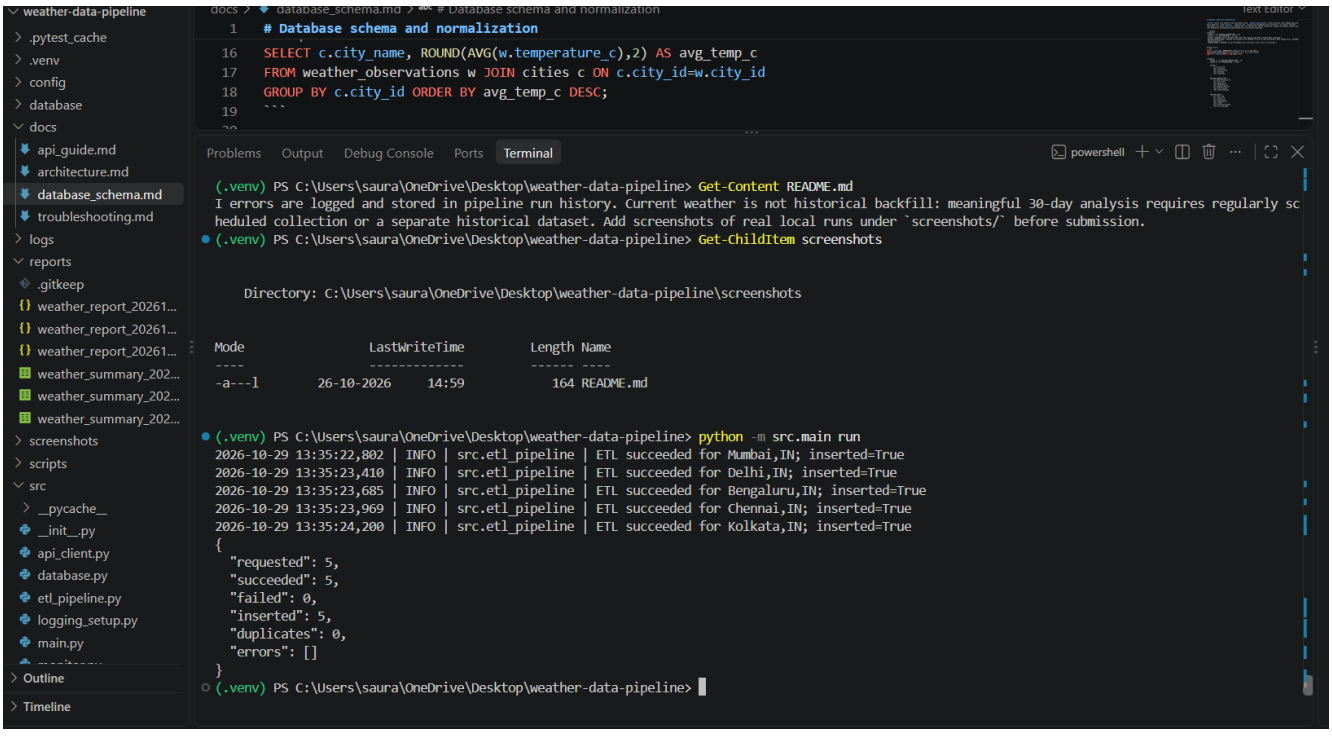


Figure B2. Automated test results

Show pytest output with five passing tests.

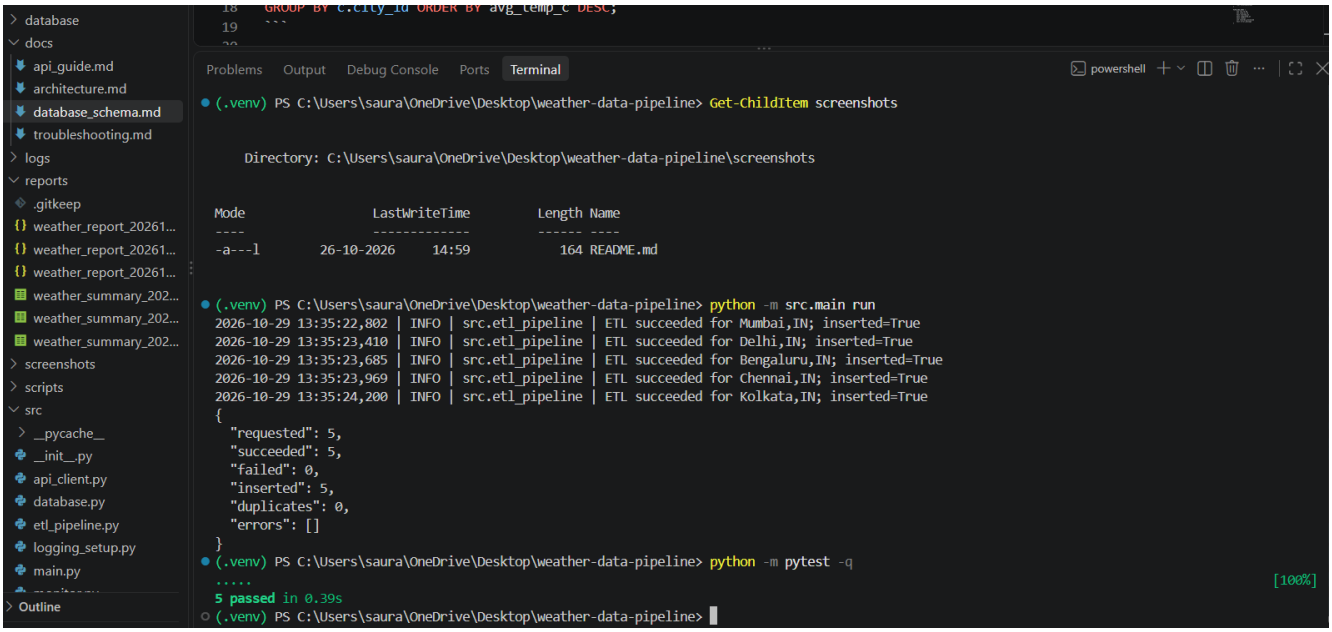


Figure B3. Pipeline health check

Show database status, city count, observation count, failed runs, freshness and healthy status.

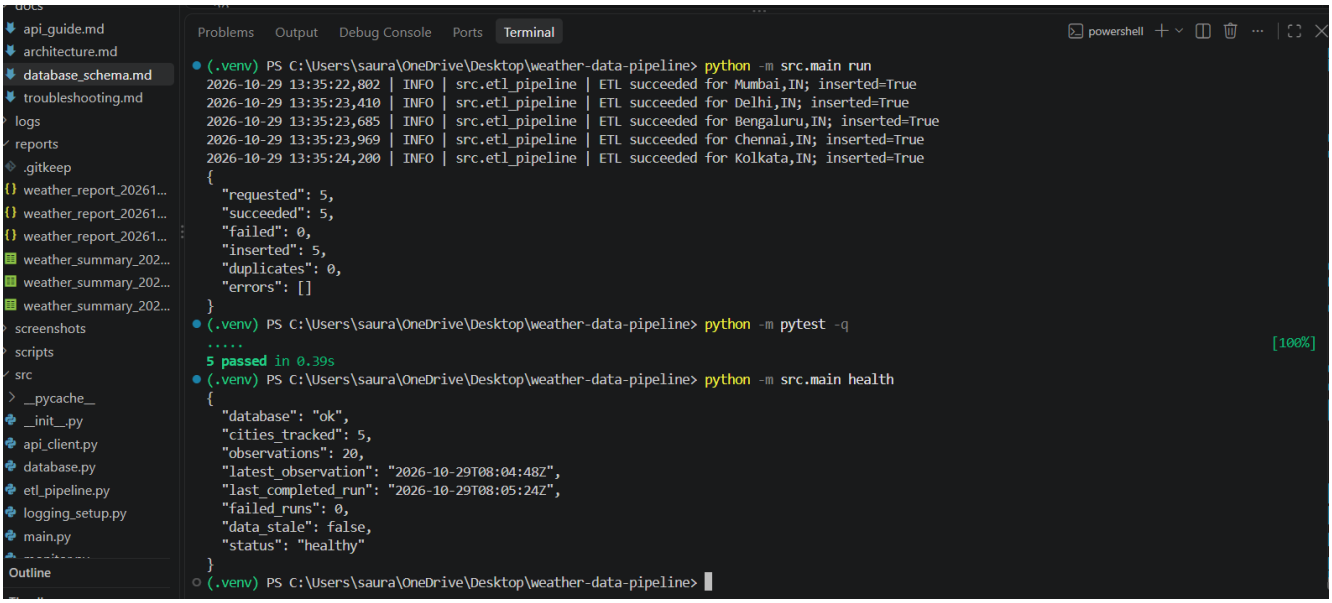
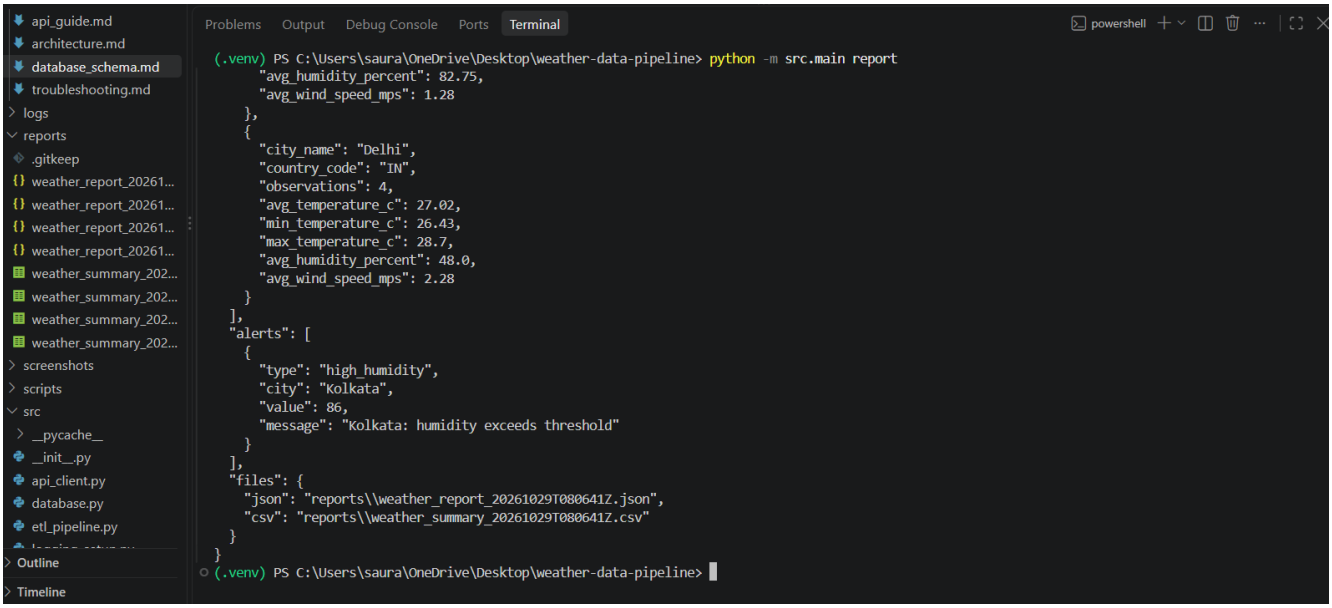


Figure B4. Generated reports

Show the generated JSON and CSV report files.



Appendix C. Assignment Requirement Mapping

Requirement	Implementation / evidence	Evidence status
SQLite with 3+ normalized tables	cities, weather_observations, pipeline_runs; keys, constraints and indexes.	Demonstrated
OpenWeatherMap API client	src/api_client.py and successful five-city run.	Demonstrated
ETL, error handling and logging	ETL, validators and logging modules.	Demonstrated
Validation and quality assurance	Timestamp/measurement checks, duplicate prevention; five tests pass.	Demonstrated
Automated scheduling	scheduler module; APScheduler installation/runtime issue.	Needs verification
Historical analysis queries	history/stats commands and report summaries.	Partially evidenced
Automated reports and alerts	JSON/CSV reports; Kolkata humidity alert at 91%.	Demonstrated
Monitoring / health checks	health command reports healthy database metrics.	Demonstrated
Documentation and screenshots	README/docs and screenshot folder; final screenshot review required.	Review before submission

End of Report